



Status Update – The model now monitors every position that is opened to determine the positions state, and what it should do and when. I have also added some cmd line args to the simulator-backtester which effectuates an optimization to the profit targets across all 58 strategies which are now fully integrated, and part of the model training. Additional cmd line args were added which enable us to include All 58 strategies, select just one options strategy, or omit specific strategies from All upon instantiation of the simulator-backtester / simulator script, e.g.:

```
git pull && python kaggle/condor_brain_backtest_v45.py --use-v43 --verbose --max-positions 6 --strategyomit "iron_condor" --profittargets > backtest_run.log 2>&1 &
```

Then to show the outputs:

```
tail -f backtest_run.log
```

Currently, I am neck deep in doing some CUDA kernel optimizations for much faster inference and simulation.

Please note you will not be able to run this script without having a model file that was recently trained using the training scripts and the datafiles which there are 5 of them – these have not been uploaded to the Github repo.

Entries with Exits and Cognitive Holds, Predictive Analytic Folds, and watch the equity grow... These have been fully integrated into the model training and model simulator (its not just a backtester, as you will see by the output it's a forensic CondorNet laboratory for monitoring absolutely everything the model does to make the determination to Enter, Hold, or Exit which when completed and debugged will bring CNv43 to CNv45 as the tasks for CNv43 to CNv44 were completed however I forgot to name them and just updated the CNv43 scripts.

The moment we move from **snapshot predictor** to **closed-loop portfolio controller**, the problem changes from classification to **sequential optimal control under capital constraints**.

What we need to integrate into CNv4.3 now, first, are as follows:

- A **position-aware state machine**
- A **hierarchical exit taxonomy**
- A **SimExit labeling engine** that defines “improve vs replicate”
- A portfolio-level flatten logic layer

Below is a structured framework I have tailored for CondorNet v43 ETD-1 part of the equation:

I. Closed-Loop Architecture (Control View)

At each M5 bar t :

The model evaluates:

1. Entry decision π_t^{enter}
2. Exit decision π_t^{exit}
3. Position sizing subject to capital constraint
4. Portfolio-level override

State now becomes:

$$S_t = [X_t^{market}, V_t^{portfolio}, M_t^{risk}]$$

Where:

- X_t^{market} = market features
 - $V_t^{portfolio} \in \mathbb{R}^{d_v}$ = position state (real, not proxy)
 - $M_t^{risk} \in \mathbb{R}^{d_m}$ = memory of prior trade regimes
-

II. Capital Constraint Engine (Portfolio Layer)

Let:

- B_t = account balance
- $\alpha = 0.30$ (max capital fraction)
- L = leverage (e.g., 2:1)

Max deployable capital:

$$C_{\max,t} = \alpha \cdot L \cdot B_t$$

Used capital:

$$C_{used,t} = \sum_{i=1}^{N_t} C_i$$

Remaining:

$$C_{avail,t} = C_{\max,t} - C_{used,t}$$

Entry allowed iff:

$$C_{strategy} \leq C_{avail,t}$$

Portfolio flatten rule:

$$\text{If } \sum_{i=1}^{N_t} \text{PnL}_i(t) > 0 \Rightarrow \text{Close All}$$

This becomes a deterministic override layer above model output.

III. Position State Vector (Critical Addition)

For each open position i :

$$P_t^{(i)} = \begin{bmatrix} \text{PnL\%} \\ \text{Unrealized \$} \\ \text{Credit received} \\ \text{Bars held} \\ \text{DTE remaining} \\ \text{Delta exposure} \\ \text{Gamma exposure} \\ \text{Theta decay} \\ \text{IV change since entry} \\ \text{High-water mark} \\ \text{Max adverse excursion} \end{bmatrix}$$

When idle:

$$P_t^{(i)} = 0$$

This feeds the v -block.

Without this, ETD-1 memory has nothing to integrate.

IV. Multi-Stage Exit Rule Taxonomy

We need structured exit classes before training.

Think in 5 layers:

1. Hard Risk Exits (Non-Negotiable)

These are guardrails — not learned.

A. Max Loss Exit

$$\text{If } \text{PnL\%} \leq -L_{max} \Rightarrow \text{Exit}$$

Example: $L_{max} = 200\%$ of credit for short premium.

B. Delta Violation Exit

$$|\Delta_{position}| > \Delta_{threshold}$$

Example: > 0.30 net delta for iron condor.

C. Capital Emergency Exit

If portfolio drawdown:

$$\frac{B_t - B_{peak}}{B_{peak}} < -D_{max}$$

These remain deterministic.

2. Profit-Taking Exits (Structured but Learnable)

A. Static Profit Target

$$PnL\% \geq T_{profit}$$

Example: 50% of credit.

B. Dynamic Profit Based on Volatility Compression

If IVR collapses:

$$IVR_t < IVR_{entry} - \kappa$$

Exit early even if not at target.

C. Theta Efficiency Exit

If:

$$\frac{d(PnL)/dt}{\text{Remaining Risk}} < \epsilon$$

Trade has “stalled”.

3. Time-Structure Exits

A. DTE Window Exit

Define hold regime:

- Early phase: $DTE > 14$
- Decay phase: $7 < DTE \leq 14$
- Gamma phase: $DTE \leq 7$

Exit logic may differ by regime.

Example:

If $DTE \leq 5$ and not at profit target $\Rightarrow Exit$

B. Bars Held Cap

Bars held $> H_{max}$

4. Market Structure Exits (Advanced — Learnable)

This is where pivots enter.

A. Structural Break Exit

If price breaks pivot range:

$Close_t > \text{Strong Pivot High}$

for short call side.

B. Momentum Reversal Exit

If:

MACD histogram crosses against position

C. Regime Flip Exit

If:

Vol regime shifts from compression to expansion

This is ideal for training improvement.

5. Portfolio-Level Exit

Collective flatten condition:

$$\sum_{i=1}^{N_t} \text{UnrealizedPnL}_i > 0$$

OR:

Portfolio VaR exceeds threshold

V. When NOT to Exit (Critical)

Model must learn “hold discipline.”

Define HOLD zones:

1. Pre-Decay Zone

If:

$$DTE > 14$$

and position within expected value band:

$$| \text{Spot} - \text{MidRange} | < \sigma_{\text{expected}}$$

Do not exit early.

2. Pivot Containment Zone

If price remains inside:

[Medium Pivot Low, Medium Pivot High]

Position still structurally valid. But have the model learn that if price is nearing a predicted pivot, and the pivot marks while still in the trade, exit all symbols where the direction change of the market defined by the pivot point is against the trade. This is not a soft gate but a hard stop to protect whatever the position has in the money, or minimize downside for however much it is already in the red.

3. Favorable Theta Zone

If:

$$\Theta > \Theta_{threshold}$$

and delta stable.

VI. SimExit Labeling Framework (The Hard Part)

For every historical entry T_0 :

At each future bar T :

Compute:

1. Exit now value:

$$V_{exit}(T)$$

2. Continue value:

$$V_{hol}(T) = \max_{T < \tau \leq expiry} V(\tau)$$

Or if using hard natural exit:

$$V_{hold}(T) = V_{natural_exit}$$

Label:

$$y_{exit}(T) = \begin{cases} 1 & \text{if } V_{exit}(T) \geq V_{hold}(T) - \epsilon \\ 0 & \text{otherwise} \end{cases}$$

This teaches improvement over hard rule.

VII. Replicate vs Improve Decision

Three training strategies:

Option A: Replicate Hard Rules

Train model to match deterministic exits.

Pros:

- Stable
- Interpretable

Cons:

- No alpha
-

Option B: Improve on Hard Rules (Recommended)

Hard exits remain safety layer.

Model learns:

$$\pi_t^{exit} = \mathbb{P}(V_{exit} > V_{ho})$$

We allow override only if confidence > threshold.

Option C: Fully Learned Exit

Remove deterministic rules.

Not recommended for first iteration.

VIII. Exit Head Design

Add:

$$\hat{p}_t^{exit} = \sigma(Wh_t + b)$$

Binary cross entropy vs SimExit label.

But more powerful:

Use regression head:

$$\widehat{\Delta V}_t = \hat{V}_{exit} - \hat{V}_{hold}$$

Exit if positive.

This avoids classification threshold issues.

IX. Integration with ETD-1

ETD-1 continuous update:

$$x_{t+1} = e^{A_\theta(u_t, \sigma_t) \Delta t} x_t + \Phi_1(\cdot) B_\theta(u_t)$$

Now u_t includes:

$$u_t = [X_t^{market}, P_t^{position}]$$

So memory block now encodes:

- Entry context
- Trade trajectory
- Regime shift
- Drawdown path

Exactly what it was architected for.

X. My Suggested Multi-Stage Exit Stack (Production Version)

1. Hard Risk Exit (always on)
2. Model Exit Probability
3. Structural Pivot Exit
4. DTE Regime Exit
5. Portfolio Flatten Override

Exit if:

$$\text{HardExit} \vee (\hat{p}_t^{exit} > 0.7 \wedge \text{Not in Protected Hold Zone})$$

XI. What Changes Everything

This converts CondorNet from:

“Predict best entry”

to

“Maximize expected trade lifecycle value under capital constraint”

That is institutional-grade control.

XII. Exit Scaffolding for CondorNet Training Scripts

Some example scripts I wrote for Claude Opus 4.6 to use as-is or as a guide to tightly weave these exits into the training python scripts v4.3

1. Add an Exit Head to the Model: In `condor_brain_net_v43.py`, we define a new ExitHead module and incorporate it into CondorNetV43. For example, after the existing output heads:

```
class ExitHead(nn.Module):
    """Exit decision head: outputs a [B,1] exit probability (sigmoid)."""
    def __init__(self, d_joint: int = 384, dropout: float = 0.1):
        super().__init__()
        self.exit_head = nn.Sequential(
            nn.Linear(d_joint, 1),
        )
    def forward(self, joint_repr: torch.Tensor) -> torch.Tensor:
        x = self.exit_head(joint_repr)
        return torch.sigmoid(x)
```

In the CondorNetV43. `__init__`, instantiate the exit head alongside others:

```
self.spot_head = nn.Sequential(
    nn.Linear(d_joint, 64),
    nn.GELU(),
    nn.Linear(64, 1),
)
self.exit_head = ExitHead(d_joint) # NEW: Exit signal head
```

And in the forward pass (after computing the joint representation `joint_last`), compute the exit signal:

```
# Existing heads
strategy_logits, leg_params, entry_signal = self.strategy_head(joint_last)
pop, ev, max_loss, var_95, cvar_95 = self.risk_head(joint_last)
pivot_high_probs, pivot_low_probs, pivot_strength = self.pivot_pred_head(joint_last)
position_size = self.size_head(pop.detach(), joint_last, self.pop_sizing_weight)
spot_pred = self.spot_head(joint_last)

# NEW: compute exit probability
exit_signal = self.exit_head(joint_last) # [B, 1] sigmoid output

return CondorNetOutput(
    strategy_logits=strategy_logits,
    leg_params=leg_params,
    entry_signal=entry_signal,
```

```
pop=pop, ev=ev, max_loss=max_loss, var_95=var_95, cvar_95=cvar_95,
pivot_high_probs=pivot_high_probs, pivot_low_probs=pivot_low_probs, pivot_strength=pivot_strength,
position_size=position_size, spot_pred=spot_pred,
exit_signal=exit_signal,      # NEW output
final_gate=core_diag.get("final_gate"),
diagnostics=diags,
consensus_strength=consensus_strength,
)
```

Also update the CondorNetOutput dataclass to include exit_signal (e.g. after entry_signal):

```
class CondorNetOutput:
    strategy_logits: torch.Tensor # [B, 10]
    leg_params:      torch.Tensor # [B, 4, 5]
    entry_signal:    torch.Tensor # [B, 1]
    exit_signal:     torch.Tensor # [B, 1] – exit probability ∈ [0,1]
    pop:             torch.Tensor # [B, 1]
    ...
```

2. Add Exit Labels to the Data Pipeline: In `data_pipeline_v43.py`, extend the multi-task label generator. In `PIPELINE_LABEL_NAMES`, append an "exit_signal" label. For example:

```
PIPELINE_LABEL_NAMES = [
    "strategy_label", "pop", "ev", "max_loss", "var_95", "cvar_95", "exit_signal"
]
```

In the `compute_multitask_labels(df, ...)` function (after computing all other targets), add a default or simulated exit label for each M5 row. For now, as a placeholder we can set:

```
df["exit_signal"] = 0.0
```

This creates an exit label (float 0.0) for every bar. In a full implementation, we would replace this with our simulated exit logic ("SimExit") to mark bars where exiting is optimal. But for integration, adding the column avoids mismatches.

3. Modify the Dataset Loader: The `V43Dataset` class reads labels from the pipeline output. Ensure it passes the new exit label into the training batch. After updating `PIPELINE_LABEL_NAMES`, the existing code in `__getitem__` will automatically include `exit_signal` in the `target_labels` dict (since it loops over all keys of `self.labels`). No further change is needed there.

4. Train Loop – Add Exit Loss: In `condor_train_net_v43.py`, update the training loss to use the exit signal. After getting `outputs = model(...)` and retrieving the batch labels, add something like:

```
exit_target = labels["exit_signal"].unsqueeze(-1) # [B,1]
components['exit_bce'] = F.binary_cross_entropy(outputs.exit_signal, exit_target)
```

You will also need to assign a weight for this component. If using a loss configuration JSON (e.g. `loss_weights_v43.json`), add an entry for "exit_bce". For example:

```
{
  "initial": {
    "strategy_ce": 1.0,
    "pop_bce": 1.0,
    ...
    "exit_bce": 1.0
  }
}
```

And in the CondorNetLoss aggregation (if one exists), include components['exit_bce'] with its weight.

5. (Optional) Position State Inputs: If we plan to feed position-state features (PnL%, bars held, etc.) into the model, extend the dataset and model inputs. For example, we could append these as additional features in `data_pipeline_v43.py` (added to `m5_features` or a separate input). In `condor_brain_net_v43.py`, increase `d_tf_in` and ensure the model's projector layers accept the extra dimensions. For now, this step is optional – the exit head can still use the “joint” representation to make decisions without explicit position-state inputs.

Summary: These changes introduce a new “exit_signal” branch to CondorNetV43. The model now outputs a probability of exit (fold) at each bar. The pipeline provides a placeholder label (0.0) for training; this can be replaced by simulated exit outcomes later. By following the above steps, we integrate multi-stage exit decision logic into the existing framework with minimal disruption.
